

Introducción a Assistant Engineering

Ahora uniremos todo lo visto anteriormente en un **producto** : un **asistente conversacional**. Como ejemplo, usaremos un asistente de estudio.

Ya no preguntamos solo “¿qué prompt envió?”. Preguntamos “¿cómo diseño el sistema que envía prompts en cada turno?”.

Assistant Engineering = diseñar un sistema Python que, en cada turno, combine configuración, estado y contexto para llamar al LLM de forma coherente y mantenible.

Objetivos de la unidad

Al terminar, deberías poder:

- Diferenciar una **llamada suelta** a Gemini de un **asistente con arquitectura**.
- Describir las **capas** de un asistente mínimo (configuración, estado, prompts, lógica, orquestación).
- Mantener **estado de sesión** y **configuración** separados del texto que escribe el usuario.
- **Personalizar** respuestas (perfiles, tono, nivel) sin reescribir todo el código.
- Implementar un **flujo conversacional** completo: input → contexto → prompt → modelo → respuesta → actualizar estado.

Qué es (y qué no es) un asistente

Sí es	No es
Un sistema que orquesta varias llamadas al LLM con memoria de sesión	Un único prompt gigante pegado en un script
Estado + configuración + prompts coordinados en cada turno	“ChatGPT en una línea de Python” sin estructura
Código mantenible (config , state , prompts , logic , main)	Copiar y pegar respuestas del modelo en variables sueltas
Evolución natural de Prompt y Context Engineering	Un framework externo (LangChain, etc.) — aquí seguimos en Python puro

Assistant Engineering aquí significa: **diseñar la arquitectura** que convierte interacciones sueltas en una experiencia conversacional coherente.

De prompt aislado a asistente estructurado

La idea central es que un asistente es un sistema que orquesta varias llamadas al LLM con memoria de sesión.

Prompt aislado

Una llamada
Sin memoria explícita
Instrucciones mezcladas
Difícil de extender

Asistente estructurado

Muchos turnos
Estado + historial
Config + prompts + contexto
Módulos con responsabilidad clara

Hilo narrativo: tutor de estudio

Seguiremos el dominio del **asistente de estudio** que ya apareció en Context Engineering, pero ahora como **producto**:

- El usuario se presenta, pregunta, pide aclaraciones.
- El asistente recuerda nombre, nivel y tema.
- La **configuración** define si responde como mentor, junior buddy o explicación senior.
- Cada turno **construye** el prompt con estado + config + contexto relevante.

Lo que hacemos en esta unidad es **subir de nivel** de abstracción.

Organización general de código del asistente conversacional

```
asistente/
├── config.py           # MODEL, límites, perfiles por defecto
├── state.py           # user_state, historial, summary (si aplica)
├── prompts.py         # build_assistant_prompt(...)
├── gemini_auth.py     # API key (proyecto)
├── gemini_client.py   # llamadas + métricas opcionales
├── logic.py           # procesar_turno(), validaciones ligeras
└── main.py           # demo / CLI mínima
```

La pieza que une todo: `build_assistant_prompt`

Con la función `build_assistant_prompt` conseguiremos crear un prompt coherente y mantenible en cada turno:

```
prompt = build_assistant_prompt(
    assistant_config=assistant_config,
    user_state=user_state,
    user_message=user_message,
    extra_context=extra_context, # FAQ, extractos, etc.
)
```

- `python prompt = build_assistant_prompt( assistant_config=assistant_config, user_state=user_state, user_message=user_message, extra_context=extra_context, # FAQ, extractos, etc. )`

```
- assistant_config – quién es el asistente, tono, límites, perfil activo.
- user_state – nombre, nivel, historial reciente, resumen de sesión.
- user_message – lo que dice el usuario en este turno.
- extra_context – datos de referencia seleccionados (herencia Context Engineering).
```

```
---
```

Convenciones

```
- Gemini en todos los ejemplos ejecutables; el patrón sirve para otros proveedores.
- Temperatura moderada-baja (0.2-0.4) para respuestas estables.
- Respuestas de la app en formato {"status": "ok"|"error", "mensaje": ..., "data": ...} cuando haya lógica de negocio.
- No subas API keys ni .env a Git.
```

```
---
```

Errores típicos que debemos evitar

1. `Guardar todo en un string` – instrucciones, historial y pregunta mezclados sin estructura.
2. `Confundir estado con configuración` – el nombre del usuario es estado; el tono por defecto del producto es config.
3. `Reenviar siempre todo el historial` – sin ventana ni resumen.
4. `Lógica de negocio dentro del prompt` – validaciones y ramas deben vivir en Python (`logic.py`).
5. `Un solo archivo de 400 líneas` – funciona en demo; se rompe al añadir perfiles o un tercer flujo.